

Projet Compilation - S6

Introduction :

Le projet consiste à écrire compilateur en utilisant un analyseur syntaxique en utilisant flex et bison fait en projet d'analyse syntaxique au S5.

Le langage analysé est le TPC, un sous langage du C, pour vérifier l'intégrité d'un programme, et ainsi regarder si celui-ci respecte bien les règles du langage.

Il a en plus la possibilité d'afficher les composantes d'un programme à analyser sous forme d'arbre (en utilisant tree.c) ou encore la table des symboles.

Fonctionnement :

Pour lancer les tests unitaires : `python3 ./src/test.py`

Pour lancer les tests unitaires sur un fichier spécifique : `python3 ./src/test.py good sem-err...`

Pour lancer spécifiquement un programme à compiler : `./bin/tpcc /test/...` puis `make nasm` puis `./ anonymous`

Gestion des options :

Pour visionner l'arbre abstrait lié au programme TPC : `-t`

Pour visionner les différentes tables des symboles : `-s`

Pour afficher une aide sur l'interface utilisateur : `-h`

Tests unitaires :

Voici la liste exhaustive des tests unitaires pour tester notre analyseur :

▫ good test :

- 01-basic-01.tpc
- 02-variable-01.tpc
- 03-variable-02.tpc
- 04-variable-03.tpc
- 05-variable-04.tpc
- 06-variable-05.tpc

- 07-variable-06.tpc
- 08-variable-07.tpc
- 09-variable-08.tpc
- 10-operator-01.tpc
- 11-operator-02.tpc
- 12-operator-03.tpc
- 13-operator-04.tpc
- 14-operator-05.tpc
- 15-operator-06.tpc
- 16-function-01.tpc
- 17-function-02.tpc
- 18-function-03.tpc
- 19-function-04.tpc
- 20-iffelse-01.tpc
- 21-iffelse-02.tpc
- 22-iffelse-03.tpc
- 23-iffelse-04.tpc
- 24-iffelse-05.tpc
- 25-iffelse-06.tpc
- 26-iffelse-07.tpc
- 27-function-05.tpc
- 28-function-06.tpc
- 29-while-01.tpc
- 30-while-02.tpc
- 31-static-01.tpc
- 32-static-02.tpc

▣ syn-err test:

- 01-comment-01.tpc
- 02-no-return_01.tpc
- 03-no-parenthesis-if-01.tpc
- 04-variable-if-01.tpc
- 05-no-parenthesis-while-01.tpc
- 06-no-semicolon-01.tpc
- 07-no-brace-01.tpc
- 08-no-brace-02.tpc
- 09-empty-declaration-01.tpc
- 10-bad-if-01.tpc
- 11-bad-declaration-01.tpc
- 12-syntax-error-01.tpc
- 13-empty-while-01.tpc
- 14-empty-if-01.tpc
- 15-too-much-parenthesis-01.tpc
- 16-bioperator-01.tpc
- 17-no-comma-01.tpc
- 18-no-semicolon-02.tpc

- 19-too-many-commas-02.tpc
- 20-no-function-body-01.tpc
- 21-too-many-declaration-01.tpc
- 22-bioperator-02.tpc
- 23-function-in-function-01.tpc

▣ sem-err test:

- 01-main-01.tpc
- 02-main-02.tpc
- 03-redefinition-01.tpc
- 04-redefinition-02.tpc
- 05-redefinition-03.tpc
- 06-redeclared-01.tpc
- 07-undeclared-01.tpc
- 08-implicite-01.tpc
- 09-too-few-arg-01.tpc
- 10-too-many-arg-01.tpc
- 11-void-value-not-ignored-01.tpc
- 12-not-function-01.tpc
- 13-void-return-value-01.tpc
- 14-non-void-return-void-01.tpc

▣ warn test:

- 01-uninitialized-01.tpc
- 02-implicite-conversion-01.tpc
- 03-implicite-conversion-02.tpc
- 04-implicite-conversion-03.tpc
- 05-implicite-conversion-04.tpc
- 06-implicite-conversion-05.tpc
- 07-implicite-conversion-06.tpc
- 08-non-void-without-return-01.tpc
- 09-control-reache-01.tpc
- 10-control-reache-02.tpc
- 11-division-by-zero-01.tpc
- 12-not-use-var-01.tpc
- 13-not-use-funct-01.tpc

▣ Liste des erreurs implémentées :

- Call object is not a function
- Void value not ignored as it ought to be
- Implicite declaration of function
- No declaration of function main

- Redefinition
- Redefinition implemented function
- Redefinition of type
- Non-void return void
- Void return non-void
- Too few argument
- Too many argument
- Undeclared

▫ Liste des warning implémentées :

- Control reaches end of non-void function
- Division by 0
- Implicite convert (int -> char)
- Not signature like main
- Uninitialized
- Unused function
- Unused set variable
- Unused variable

Choix faits sur le projet :

Nous avons fait le choix d'implémenter le pré-calcul des expressions constantes, notamment utile pour vérifier les divisions par 0 ou vérifier la bonne atteinte de l'instruction return dans les différents blocs.

Difficultés rencontrées :

Nous avons parfois eu du mal à trouver les erreurs de sémantique, notamment sur les codes de sortie. Il était quelquefois difficile de bien différencier les erreurs des avertissements et des confusions sur la consigne se faisaient ressentir.

Répartition du travail :

Paul a fait la partie sémantique ainsi qu'une partie de l'écriture en assembleur.
Elodie a fait une partie de l'écriture en assembleur et le compte rendu.

Conclusion :

En conclusion, nous avons un compilateur permettant de traduire les programmes en TPC, en passant par l'analyseur syntaxique fait au semestre précédent. Le projet a été enrichissant et nous a permis de mieux comprendre le fonctionnement derrière les compilateurs déjà existants et les programmes écrits. De plus, il était intéressant d'observer le lien créé entre l'analyseur syntaxique et ce projet.